Implementación de prácticas DevOps y automatización del desarrollo mediante un sistema web de gestión de tareas

Pamela Valverde Castro, Laura Segreda Elizondo, Maria José Valverde Peña, Moisés Córdoba Bonilla

Licenciatura de Ingeniería en Sistemas de Computación
Universidad Fidélitas
San José, Costa Rica
pvalverde70948@ufide.ac.cr,
lsegreda70158@ufide.ac.cr,
mvalverde00343@ufide.ac.cr,
mcordoba60995@ufide.ac.cr

***Abstract* — In Software Engineering, code integration, software testing, and application deployment are essential activities for ensuring the quality and continuity of development processes. However, when these tasks depend on manual procedures, errors, delays, and inconsistencies may arise, affecting the efficiency and reliability of software products. In response to this problem, this project aims to investigate how DevOps practices, continuous integration, continuous delivery, and automation contribute to improving the software development life cycle. As a practical proposal, the project involves developing TaskFlow DevOps, a web-based system for task management and tracking, whose development process will incorporate automated mechanisms for code integration, software testing, and deployment. The proposed methodology includes requirements analysis, application design and implementation, and functionality validation through software testing. The adoption of these practices is expected to support early error detection, reduce manual intervention in repetitive activities, and strengthen collaboration among team members. Therefore, this project seeks to demonstrate the importance of DevOps as an approach to improving the efficiency, quality, and reliability of software development processes.**

Keywords— DevOps, development automation, continuous integration, continuous delivery, automated testing, software quality.

I. INTRODUCCIÓN

El desarrollo de software constituye una actividad fundamental en la transformación digital de las organizaciones, debido a que permite construir soluciones tecnológicas orientadas a optimizar procesos, administrar información y responder a las necesidades de los usuarios. No obstante, el incremento en la complejidad de las aplicaciones y la necesidad de incorporar nuevas funcionalidades de manera frecuente representan desafíos para los equipos responsables de su desarrollo y mantenimiento. En este contexto, resulta necesario adoptar prácticas que permitan fortalecer la colaboración, mejorar la calidad del código y establecer procedimientos confiables para la integración, validación y entrega de los productos de software.

Dentro de este panorama, DevOps surge como un enfoque que busca integrar las actividades de desarrollo y operaciones mediante una cultura de colaboración, responsabilidad compartida, automatización y mejora continua. Su aplicación promueve la coordinación entre los participantes del ciclo de vida del software y favorece la implementación de mecanismos que permiten ejecutar actividades técnicas de manera estandarizada. Entre sus prácticas más relevantes se encuentran la integración continua, la entrega continua y la automatización de pruebas, las cuales contribuyen a mejorar la eficiencia de los procesos de construcción y publicación de aplicaciones [1].

La integración continua, conocida como *Continuous Integration* (CI), consiste en incorporar frecuentemente los cambios realizados por los desarrolladores en un repositorio compartido, ejecutando verificaciones automatizadas para identificar posibles defectos o conflictos de integración. Por su parte, la entrega continua, denominada *Continuous Delivery* (CD), permite preparar y validar los cambios del software mediante

procedimientos automatizados, de manera que las nuevas versiones puedan publicarse de forma controlada. Ambas prácticas buscan fortalecer la confiabilidad del proceso de desarrollo y facilitar la identificación temprana de errores [2].

A pesar de las ventajas asociadas con estos mecanismos, los proyectos que dependen de procedimientos manuales pueden enfrentar dificultades relacionadas con la integración de componentes, la ejecución inconsistente de pruebas, los errores de configuración y los retrasos en la entrega de nuevas versiones. Estas condiciones pueden afectar la productividad de los equipos, incrementar el esfuerzo necesario para detectar defectos y comprometer la estabilidad de las aplicaciones. Por esta razón, la automatización representa una alternativa para reducir actividades repetitivas y establecer mecanismos de verificación sistemáticos durante el desarrollo del software.

En respuesta a esta problemática, el presente proyecto propone investigar la aplicación de prácticas DevOps y automatización mediante el desarrollo de TaskFlow DevOps, un sistema web orientado a la gestión y seguimiento de tareas de proyectos de software. La aplicación permitirá administrar actividades, asignar responsables, establecer prioridades y visualizar el estado de las tareas. Sin embargo, el propósito principal del proyecto no se limita a la construcción de estas funcionalidades, sino que contempla utilizar su proceso de desarrollo como escenario práctico para implementar integración continua, pruebas automatizadas y procedimientos de despliegue.

Para apoyar la implementación de estos procesos, se propone utilizar herramientas de control de versiones y automatización, como GitHub y GitHub Actions. Esta última permite configurar flujos de trabajo que se ejecutan ante determinados eventos del repositorio, facilitando la automatización de actividades relacionadas con la construcción, las pruebas y el despliegue de aplicaciones [3]. De esta manera, se busca establecer un proceso de desarrollo estructurado que permita analizar los beneficios de aplicar estas prácticas en un proyecto académico de Ingeniería de Software.

La relevancia de esta investigación radica en su contribución al estudio de estrategias orientadas a mejorar la calidad del software, reducir la dependencia de procedimientos manuales y fortalecer la colaboración entre los integrantes de los equipos de desarrollo. Asimismo, permite relacionar los fundamentos teóricos de DevOps con una experiencia práctica, mediante la cual será posible examinar los resultados de su implementación, identificar limitaciones y valorar su utilidad en el ciclo de vida de una aplicación.

El artículo se estructura mediante la presentación del problema de investigación, los objetivos generales y específicos, el marco teórico relacionado con DevOps y la automatización, la metodología propuesta y el análisis de requerimientos. Posteriormente, se abordarán el diseño, la arquitectura, la implementación, las pruebas y los resultados del sistema desarrollado, con el propósito de evaluar el cumplimiento de los objetivos y formular las conclusiones correspondientes.

II. PLANTEAMIENTO DEL PROBLEMA

En los procesos de desarrollo de software, la integración de cambios, la ejecución de pruebas y el despliegue de aplicaciones constituyen actividades esenciales para garantizar que los productos tecnológicos funcionen de acuerdo con los requerimientos establecidos. Sin embargo, cuando estas actividades se realizan mediante procedimientos manuales y sin mecanismos de automatización, pueden producirse errores, retrasos e inconsistencias que afectan la eficiencia del equipo y la calidad de las entregas. Esta situación representa una problemática relevante en proyectos donde múltiples desarrolladores trabajan de manera colaborativa y realizan modificaciones frecuentes sobre una misma aplicación.

Una de las principales dificultades se encuentra en la dependencia de intervenciones manuales para verificar los cambios incorporados al código fuente. Cuando los desarrolladores integran sus modificaciones sin un proceso automatizado de validación, existe el riesgo de introducir defectos que no sean identificados oportunamente. Asimismo, la ejecución manual de pruebas puede generar inconsistencias en los procedimientos de verificación, debido a que las comprobaciones realizadas pueden variar según la persona responsable o las condiciones del entorno. Estas circunstancias dificultan la detección temprana de errores y aumentan el esfuerzo necesario para mantener el correcto funcionamiento del software.

De manera similar, la ausencia de procedimientos estandarizados para el despliegue representa una dificultad adicional, ya que la publicación de nuevas versiones puede depender de configuraciones realizadas manualmente. Esto incrementa el riesgo de errores operativos, diferencias entre ambientes y problemas de compatibilidad. De acuerdo con las prácticas de integración y entrega continua, la incorporación de mecanismos automatizados permite establecer procedimientos repetibles para construir, verificar y preparar nuevas versiones de una aplicación [2].

Los principales actores afectados por esta problemática son los desarrolladores de software, responsables de aseguramiento de la calidad, encargados de operaciones y coordinadores de proyectos tecnológicos. Los desarrolladores deben invertir tiempo en actividades repetitivas de integración y verificación; los responsables de calidad pueden enfrentar dificultades para garantizar la ejecución consistente de las pruebas; mientras que los encargados de operaciones deben gestionar los riesgos relacionados con la publicación y estabilidad de las aplicaciones. Como consecuencia, la falta de automatización puede limitar la capacidad del equipo para mantener un proceso de desarrollo organizado, verificable y confiable.

En el contexto del presente proyecto, esta problemática se abordará mediante el desarrollo de TaskFlow DevOps, una aplicación web destinada a la gestión y seguimiento de tareas de proyectos de software. La aplicación funcionará como un escenario práctico para analizar e implementar mecanismos de automatización durante su ciclo de vida. Para ello, se propone establecer un repositorio compartido de código fuente, configurar procedimientos de integración continua, ejecutar pruebas automatizadas y definir un flujo de entrega y despliegue que permita verificar los cambios antes de su publicación.

Es importante destacar que el problema central de la investigación no corresponde a la ausencia de herramientas para administrar tareas, debido a que existen diferentes soluciones destinadas a esta finalidad. Por el contrario, la problemática se concentra en las dificultades que pueden surgir cuando los procesos de integración, pruebas y despliegue dependen de actividades manuales y carecen de mecanismos estandarizados de automatización. En este sentido, TaskFlow DevOps constituye el medio para aplicar los conocimientos investigados y evaluar la utilidad de las prácticas DevOps en un entorno de desarrollo académico.

La necesidad que se pretende atender consiste en establecer un proceso de desarrollo de software que permita integrar cambios de manera controlada, ejecutar verificaciones automáticas y facilitar la preparación y entrega de nuevas versiones. Mediante estas prácticas, se busca reducir la intervención manual en actividades repetitivas, favorecer la detección temprana de defectos y fortalecer la confiabilidad del proceso de desarrollo. Además, se pretende promover la colaboración y la responsabilidad compartida entre los integrantes del equipo, aspectos fundamentales del enfoque DevOps.

El alcance del proyecto se delimita al diseño, desarrollo y validación de una aplicación web de gestión de tareas, acompañada de un flujo básico de integración continua, pruebas automatizadas y entrega de software. La investigación se desarrollará en un entorno académico, por lo que no pretende implementar una infraestructura empresarial de gran escala ni evaluar todos los componentes del enfoque DevOps. Su propósito será demostrar la aplicación de prácticas seleccionadas y analizar su contribución a la eficiencia y calidad del proceso de desarrollo.

Por lo tanto, el problema específico que orienta la investigación se define como la dependencia de procedimientos manuales y la falta de mecanismos automatizados para integrar, probar y desplegar cambios en una aplicación de software, situación que puede generar errores, inconsistencias y retrasos durante su ciclo de vida.

A partir de lo anterior, se establece la siguiente pregunta de investigación:

¿Cómo contribuye la implementación de prácticas DevOps, mediante la integración continua, la entrega continua y la automatización de pruebas, a mejorar la eficiencia, calidad y confiabilidad del proceso de desarrollo de la aplicación web TaskFlow DevOps?

III. OBJETIVOS

Objetivo General:

Implementar prácticas DevOps y mecanismos de automatización del desarrollo mediante la construcción de un sistema web de gestión de tareas denominado TaskFlow DevOps, utilizando herramientas de integración continua, pruebas automatizadas y entrega de software, con el propósito de evaluar su contribución a la eficiencia, calidad y confiabilidad del proceso de desarrollo en un entorno académico.

Objetivos Específicos:

- Desarrollar el sistema web TaskFlow DevOps mediante Python, Flask, SQLite, HTML, CSS y JavaScript, incorporando funcionalidades para la creación, asignación, clasificación, priorización y seguimiento de tareas de proyectos de software.

- Implementar un flujo automatizado de integración continua mediante GitHub y GitHub Actions, que permita ejecutar verificaciones y pruebas automatizadas del sistema web TaskFlow DevOps para

estandarizar la validación del código fuente y facilitar la preparación de nuevas versiones.

- Evaluar la implementación de las prácticas DevOps mediante el análisis de los resultados de las pruebas automatizadas y de las ejecuciones del flujo de integración continua, con el fin de identificar sus contribuciones y limitaciones en términos de eficiencia y confiabilidad del proceso de desarrollo de TaskFlow DevOps.

IV. MARCO TEÓRICO

- **DevOps en el desarrollo de software.**

DevOps es un enfoque que integra las actividades de desarrollo de software y operaciones mediante la colaboración entre los equipos, la automatización de procesos y la mejora continua. Su propósito consiste en facilitar la construcción, validación y entrega de aplicaciones de manera más eficiente y confiable, promoviendo así la responsabilidad compartida durante las diferentes etapas del ciclo de vida del software. Según Microsoft, DevOps combina personas, procesos y tecnología para mejorar la coordinación entre las actividades de planificación, desarrollo, entrega y operación de aplicaciones [4].

La implementación de DevOps requiere adoptar prácticas que permitan reducir la dependencia de procedimientos manuales y establecer procesos repetibles para verificar los cambios realizados en el código fuente. Entre estas prácticas destacan el control de versiones, la entrega continua y la automatización de pruebas. Su uso permite detectar defectos tempranos, mantener criterios de validación consistentes y hacer más sencilla la preparación de nuevas versiones del software.

En el proyecto que se desea realizar de TaskFlow DevOps, se llevará a cabo este enfoque mediante el desarrollo de una aplicación web para administrar tareas de proyectos de software y la configuración de un flujo automatizado. De esta forma, el proyecto va a permitir estudiar la aplicación práctica de DevOps en un entorno académico, relacionando el desarrollo de funcionalidades con la automatización de las verificaciones del código y el análisis de los resultados obtenidos.

- **Automatización del desarrollo mediante GitHub Actions**

La automatización del desarrollo consiste en usar herramientas que ejecutan actividades técnicas de manera programada o en respuesta a eventos específicos, logrando minimizar la necesidad de intervención manual. En los procesos DevOps, la automatización puede aplicarse a la instalación de dependencias, las verificaciones del código, la ejecución de pruebas y la preparación de versiones.

GitHub Actions es una plataforma de automatización que permite definir flujos de trabajo mediante archivos de configuración en formato YAML, almacenados en el repositorio, estos flujos se componen de eventos que los activan, trabajos (jobs) y pasos (steps) que ejecutan comandos o acciones reutilizables [5].

En TaskFlow DevOps, GitHub Actions se usará para configurar un flujo de integración continua que ejecute las pruebas automatizadas desarrolladas con Pytest. El flujo podrá activarse, por ejemplo, al realizar una modificación en el repositorio o al crear una solicitud de integración (pull request). De esta manera, el equipo podrá conocer si los cambios superan las verificaciones establecidas antes de permitir su incorporación a la rama principal.

La configuración del flujo también permitirá aplicar reglas de protección sobre las ramas, de modo que los cambios deban superar las verificaciones requeridas antes de ser fusionados. Esta medida se relaciona con la regla de negocio RN03 del proyecto (mencionada más adelante en el artículo), que establece que los cambios deben superar las pruebas automatizadas antes de integrarse en la rama principal o desplegarse. Es importante distinguir que la ejecución de las pruebas y el bloqueo efectivo de una integración son mecanismos diferentes: para impedir una fusión cuando las pruebas fallan, es necesario configurar las políticas correspondientes en el repositorio.

- **Integración y entrega continuas (CI/CD)**

La integración continua (CI) es una práctica de desarrollo que consiste en integrar frecuentemente los cambios realizados por los programadores en un repositorio compartido y ejecutar procesos automatizados para comprobar que el código mantiene las condiciones de calidad establecidas. Microsoft señala que la integración continua permite automatizar la construcción y las pruebas del código cada vez que se incorporan cambios al sistema de control de versiones, favoreciendo la detección temprana de defectos [6].

Por otra parte, la entrega continua (CD) consiste en automatizar el progreso de los cambios en el código de software a través de pruebas y preparar el software para su publicación en los entornos que corresponda [7]. Esta práctica busca que las nuevas versiones puedan entregarse

de forma controlada y reproducible. Aunque la integración continua y la entrega continua están relacionadas, no son equivalentes: la primera se concentra en integrar y validar los cambios, mientras que la segunda amplía el proceso hacia la preparación de versiones entregables.

Para TaskFlow DevOps, la integración continua permitirá ejecutar verificaciones y pruebas automatizadas cada vez que se produzcan determinados eventos en el repositorio. Los resultados de estas ejecuciones servirán para comprobar si los cambios cumplen los criterios definidos antes de ser incorporados a la rama principal. Si el alcance del proyecto incluye la preparación automatizada de versiones, también será posible analizar cómo este proceso contribuye a organizar la entrega del sistema.

- **Control de versiones con Git y GitHub**

El control de versiones es un mecanismo que registra los cambios realizados sobre los archivos de un proyecto a lo largo del tiempo, de modo que sea posible consultar versiones anteriores, comparar modificaciones y recuperar un estado previo del código. Git es un sistema de control de versiones distribuido: cada colaborador dispone de una copia completa del repositorio con todo su historial, lo que facilita el trabajo en paralelo mediante ramas (*branches*) que posteriormente se integran en una rama principal [8].

GitHub es una plataforma que aloja repositorios Git y agrega mecanismos de colaboración, como las solicitudes de integración (*pull requests*) y la revisión de cambios. En TaskFlow DevOps, el repositorio compartido será el punto de partida del flujo automatizado: cada funcionalidad se desarrollará en una rama propia y se integrará a la rama principal mediante un *pull request*, evento que activará las verificaciones configuradas en GitHub Actions.

- **Pruebas automatizadas y Pytest**

Las pruebas automatizadas son programas que verifican, sin intervención manual, que el software se comporte según lo esperado. Al poder ejecutarse de forma repetida y con los mismos criterios cada vez que cambia el código, reducen las inconsistencias propias de la verificación manual y resultan adecuadas para integrarse a un flujo de integración continua. Entre los tipos de pruebas más usuales se encuentran las pruebas unitarias, que validan una unidad pequeña del código de forma aislada, y las pruebas de integración, que verifican la interacción entre componentes.

Pytest es un *framework* de pruebas para Python que permite escribir pruebas con una sintaxis sencilla basada en aserciones (*assert*), descubrir automáticamente los archivos y funciones de prueba, y reutilizar configuraciones mediante *fixtures* [9]. En TaskFlow DevOps se utilizará para validar la lógica de la aplicación Flask y el cumplimiento de las reglas de negocio (RN01 a RN04). Estas pruebas se ejecutarán en GitHub Actions; un resultado fallido indicará que el cambio no cumple los criterios establecidos y, mediante la protección de la rama principal, no deberá integrarse (RN03).

- **Metodologías ágiles y Scrum**

Las metodologías ágiles agrupan enfoques de desarrollo basados en ciclos cortos, entregas incrementales, retroalimentación constante de los interesados y capacidad de adaptación al cambio. Scrum es un marco de trabajo ágil que organiza el desarrollo en iteraciones de duración fija, denominadas *sprints*, de un mes o menos [10].

Scrum define tres responsabilidades dentro del equipo (*Product Owner*, *Scrum Master* y *Developers*), cinco eventos (el *sprint*, la planificación del *sprint*, el *Daily Scrum*, la revisión del *sprint* y la retrospectiva) y tres artefactos (*product backlog*, *sprint backlog* e incremento) [10]. Este enfoque se relaciona con DevOps porque ambos promueven entregas frecuentes, colaboración y retroalimentación temprana; en TaskFlow DevOps, la integración continua permitirá verificar de forma automática el incremento generado en cada *sprint*.

V. METODOLOGÍA

La metodología establece el modelo de proceso, las etapas, la organización del equipo, las herramientas, la forma de obtener los requerimientos y la estrategia de validación con la que se desarrollará TaskFlow DevOps.

- **Modelo de proceso seleccionado y justificación**

El desarrollo seguirá un modelo ágil basado en Scrum, con *sprints* de dos semanas [10]. Esta elección se justifica por tres razones. Primero, los requerimientos podrán refinarse a medida que se construye el sistema, mientras que un modelo en cascada concentra la verificación al final y retrasa la retroalimentación, lo que contradice el objetivo de detectar errores de forma temprana. Segundo, Scrum y DevOps comparten las entregas frecuentes, la colaboración y la retroalimentación continua: cada *sprint* produce un incremento que se valida de forma automática en el flujo de integración continua. Tercero, los ciclos

cortos se ajustan a un equipo de cuatro integrantes y al calendario académico, ya que permiten contar con evidencia de avance antes de cada entrega parcial.

- **Etapas del proceso**

Etapa 1. Inicio y análisis de requerimientos. Se define el problema, el alcance, los requerimientos, las reglas de negocio y los casos de uso, y se conforma el *product backlog* inicial.

Etapa 2. Planificación del sprint. Se seleccionan los elementos del *product backlog* según su prioridad y se definen las tareas del *sprint backlog*.

Etapa 3. Desarrollo. Cada funcionalidad se implementa en una rama propia y se integra a la rama principal mediante un *pull request*.

Etapa 4. Integración y verificación automatizada. GitHub Actions ejecuta las pruebas de Pytest ante cada *push* o *pull request*, y su resultado determina si el cambio puede integrarse (RN03).

Etapa 5. Revisión y retrospectiva. Se presenta el incremento, se valida contra los criterios de aceptación y se ajusta el *product backlog* con base en lo aprendido.

Etapa 6. Evaluación de resultados. Al finalizar, se analizan las ejecuciones del flujo de integración continua y de las pruebas para contrastarlas con los objetivos del proyecto.

- **Organización del equipo**

El equipo está conformado por cuatro integrantes que participan en el desarrollo y las pruebas (*Developers*). Dos de ellos asumen además los roles de *Product Owner*, responsable de priorizar el *product backlog* y representar a los interesados, y de *Scrum Master*, encargado de facilitar el proceso y eliminar obstáculos. La asignación de los roles se acordará entre los integrantes al iniciar el primer *sprint*. Dado el contexto académico, el profesor del curso actúa como interesado principal del proyecto. Cada *sprint* contempla una reunión de planificación al inicio, reuniones breves de seguimiento y una revisión con retrospectiva al cierre.

- **Herramientas y tecnologías**

Desarrollo. El backend se implementará en Python con Flask, un *framework* ligero adecuado para una aplicación de alcance acotado; los datos se almacenarán en SQLite, una base de datos relacional que no requiere servidor y facilita la configuración del entorno; y la interfaz se construirá con HTML, CSS y JavaScript.

Control de versiones y automatización. Se utilizarán Git y GitHub para el control de versiones y la colaboración [8], GitHub Actions para el flujo de integración continua y Pytest para las pruebas automatizadas [9].

Gestión del trabajo. El *product backlog* y el tablero del *sprint* se administrarán con GitHub Projects, donde los requerimientos se registran como *issues* enlazados con los *pull requests* correspondientes.

- **Obtención de requerimientos**

Al tratarse de un proyecto académico sin cliente externo, los requerimientos se definen a partir del análisis de herramientas existentes de gestión de tareas como referencia y se acuerdan entre los integrantes del equipo. Se priorizan y se registran en el *product backlog*, y se detallan mediante casos de uso con sus criterios de aceptación.

- **Estrategia de validación y evaluación**

La verificación del código se realizará mediante pruebas automatizadas de Pytest ejecutadas por GitHub Actions ante cada *push* y *pull request*, junto con la protección de la rama principal para que solo se integren los cambios que superen dichas pruebas (RN03). La validación de las funcionalidades se hará en la revisión de cada *sprint*, comparando el incremento con los criterios de aceptación definidos.

Para evaluar la contribución de las prácticas DevOps, se registrarán los siguientes indicadores: el porcentaje de ejecuciones exitosas del flujo de integración continua, la cantidad de fallos detectados por las pruebas antes de integrar los cambios a la rama principal, el tiempo promedio de ejecución del flujo y el número de pruebas automatizadas definidas.

VI. REQUERIMIENTOS FUNCIONALES Y NO FUNCIONALES

Para el desarrollo del sistema web TaskFlow DevOps, se han definido los siguientes requerimientos con el objetivo de delimitar el alcance del proyecto y asegurar que la aplicación cumpla tanto con las necesidades operativas de la gestión de tareas como las prácticas de automatización requerida.

Requerimientos Funcionales (RF):

- **RF01:** El sistema debe permitir a los usuarios crear, editar y eliminar tareas dentro del tablero principal.

- **RF02:** El sistema debe permitir la asignación de responsables específicos a cada una de las tareas registradas.
- **RF03:** El sistema debe posibilitar la clasificación de las tareas en tres estados distintos: pendientes ("To Do"), en progreso ("In progress") o finalizadas ("Done").
- **RF04:** El sistema debe permitir establecer niveles de prioridad (Alta, Media, Baja) y fechas límite de entrega para cada tarea**.**
- **RF05:** El sistema debe mostrar el avance del proyecto mediante un tablero visual interactivo que organice las tareas según su estado actual.

Requerimientos No Funcionales (RNF):

- **RNF01 (Arquitectura y Backend):** El backend de la aplicación debe estar desarrollada en lenguaje Python utilizando el framework Flask.
- **RNF02 (Persistencia de Datos):** La información del sistema debe almacenarse utilizando una base de datos relacional ligera, específicamente SQLite.
- **RNF03 (Interfaz de Usuario):** El frontend debe construirse utilizando HTML, CSS y JavaScript, garantizando una interfaz clara para la gestión visual.
- **RNF04 (Automatización CI/CD):** El proyecto debe contar con flujos de trabajo (pipelines) configurados en GitHub Actions para ejecutar verificaciones automáticas de integración continua.
- **RNF05 (Pruebas Automatizadas):** El sistema debe incorporar un conjunto de pruebas automatizadas desarrolladas con Pytest, las cuales deben ejecutarse de forma obligatoria antes de cualquier despliegue.

VII. REGLAS DE NEGOCIO

Las reglas de negocio establecen las normativas internas que aseguran la consistencia de los datos y el correcto flujo de trabajo dentro de la metodología DevOps aplicada al proyecto.

- **RN01(Asignación única):** Para mantener la claridad en la responsabilidad, una tarea solo puede tener un único usuario asignado como responsable principal a la vez.
- **RN02 (Obligatoriedad de Datos):** No se podrá registrar una tarea nueva si no cuenta con un título, una fecha de entrega y un nivel de prioridades establecido.
- **RN03(integración de Código):** Ningún cambio o nueva funcionalidad en el código fuente podrá ser fusionado a la rama principal (main) ni desplegado a producción si no supera previamente las pruebas automatizadas de Pytest ejecutadas mediante GitHub Actions.
- **RN04 (Trazabilidad de Estados):** Una tarea debe ser movida obligatoriamente al estado "In progress" antes de poder ser marcada como "Done", evitando saltos de estado que afecten el seguimiento del avance real.

VIII. CASO DE USO

A continuación, se detalla uno de los casos de uso principales del sistema, el cual incluye sus criterios de aceptación y priorización según la metodología de diseño.

Casos de uso: CU-01 – Crear una nueva tarea.

- **Actor Principal:** Usuario (Miembro del equipo de desarrollo)
- **Descripción:** El usuario interactúa con la aplicación web para registrar una nueva actividad técnica que debe ser completada en el ciclo de desarrollo.
- **Priorización:** Alta (Esencial para el funcionamiento básico del sistema).
- **Precondiciones**: El usuario debe tener acceso a la interfaz principal (Tablero) de TaskFlow DevOps.
- **Flujo Principal:**
  1. El usuario hace clic en la opción para añadir una nueva tarea.
  2. El sistema despliega un formulario de registro.
  3. El usuario completa los campos requeridos: nombre de la tarea, responsable asignado, fecha de entrega y prioridad.
  4. El usuario envía el formulario para guardar la tarea.
  5. El sistema procesa los datos y los almacena en la base de datos SQLite.
  6. El sistema actualiza el tablero y muestra la nueva tarea ubicada automáticamente en la columna de estado "To Do".

- **Flujo Alternativos:**
  - Paso 4ª: Si el usuario omite algún campo obligatorio (ej. prioridad), el sistema detiene el proceso y muestra un

mensaje de error solicitando que se complete la información.

- **Criterios de Aceptación:**
    a. La tarea debe ser visible inmediatamente en el tablero visual bajo la columna “To Do” sin necesidad de recargar la pagina completa.
    b. En la tarjeta generada deben visualizarse claramente mediante etiquetas o colores el responsable asignado, la fecha límite y s la prioridad es Alta, Media o Baja.

REFERENCIAS

[1] Google Cloud, “Capacidades de DevOps,” Google Cloud Documentation. [En línea]. Disponible: https://docs.cloud.google.com/architecture/devops?hl=es-419. [Consultado: 14-oct-2026].

[2] Amazon Web Services, “What is continuous integration and continuous delivery/deployment?,” AWS Documentation. [En línea]. Disponible: https://docs.aws.amazon.com/whitepapers/latest/practicing-continuous-integration-continuous-delivery/what-is-continuous-integration-and-continuous-deliverydeployment.html. [Consultado: 14-oct-2026].

[3] GitHub, “Understanding GitHub Actions,” GitHub Docs. [En línea]. Disponible: https://docs.github.com/en/actions/get-started/understand-github-actions. [Consultado: 14-oct-2026].

[4] What is DevOps? - Azure DevOps. (2023, 24 de enero). Microsoft Learn: Build with answers in reach. https://learn.microsoft.com/en-us/devops/what-is-devops

[5] Workflows - GitHub Docs. (s.f.). GitHub Docs. https://docs.github.com/en/actions/concepts/workflows-and-actions/workflows

[6] Use continuous integration - Azure DevOps. (2022, 28 de noviembre). Microsoft Learn: Build with answers in reach. https://learn.microsoft.com/en-gb/devops/develop/what-is-continuous-integration

[7] ¿Qué es la entrega continua? | IBM. (2025, 28 de mayo). IBM. https://www.ibm.com/mx-es/think/topics/continuous-delivery

[8] S. Chacon y B. Straub, "Pro Git," 2.ª ed., Apress, 2014. [En línea]. Disponible: https://git-scm.com/book/en/v2. [Consultado: 18-oct-2026].

[9] pytest-dev, "pytest documentation," pytest. [En línea]. Disponible: https://docs.pytest.org/en/stable/. [Consultado: 18-oct-2026].

[10] K. Schwaber y J. Sutherland, "The Scrum Guide," Scrum Guides, nov. 2020. [En línea]. Disponible: https://scrumguides.org/scrum-guide.html. [Consultado: 18-oct-2026].